Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 2b - Recipes for Decision-Making

Sarah Keren

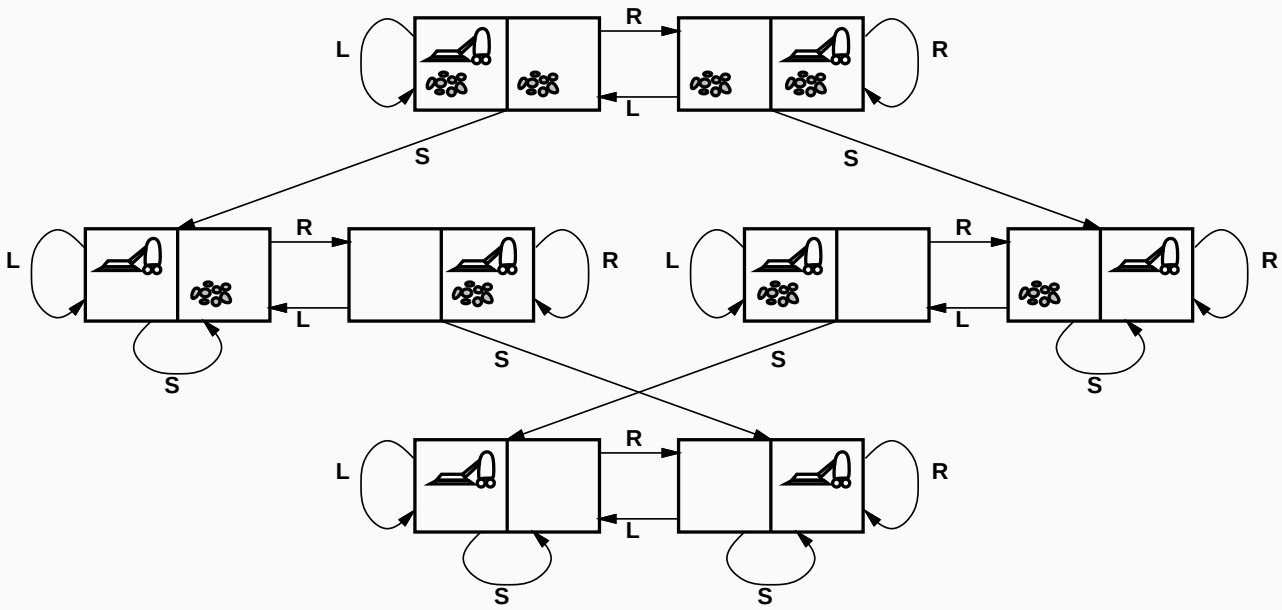
The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

So far, we have discussed models of agency.

How to find an optimal / good policy?

Examples

Example: vacuum world state space graph

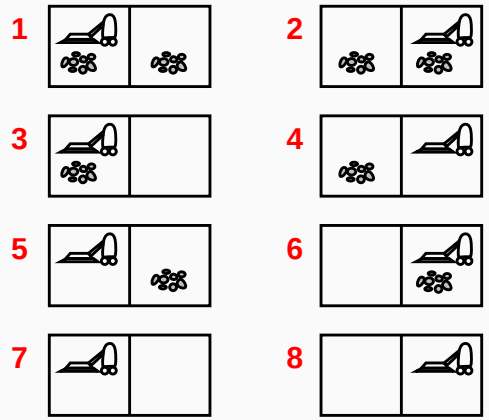


states: 0/1 dirt and robot locations (ignore dirt amounts etc.)
actions: $Left$, $Right$, $Suck$, $NoOp$
goal test: no dirt
path cost: 1 per action (0 for $NoOp$)

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Vacuum world: different classes of problems

Single-state, start in #5. Solution?



Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

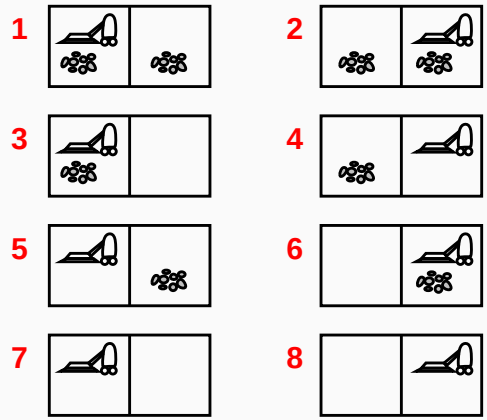
Evaluating Decision-Making Algorithms

Use Cases

Vacuum world: different classes of problems

Single-state, start in #5. Solution?

$[Right, Suck]$



Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

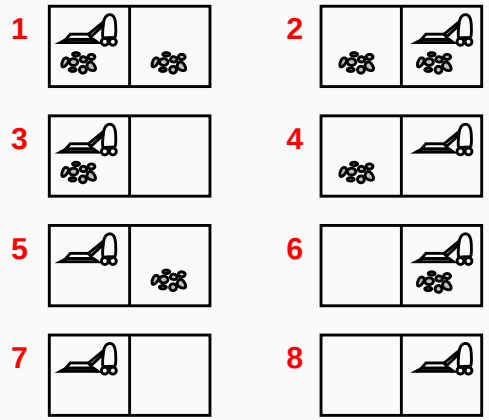
Vacuum world: different classes of problems

Single-state, start in #5. Solution?

$[Right, Suck]$

Conformant, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$

e.g., $Right$ goes to $\{2, 4, 6, 8\}$. Solution?



Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Vacuum world: different classes of problems

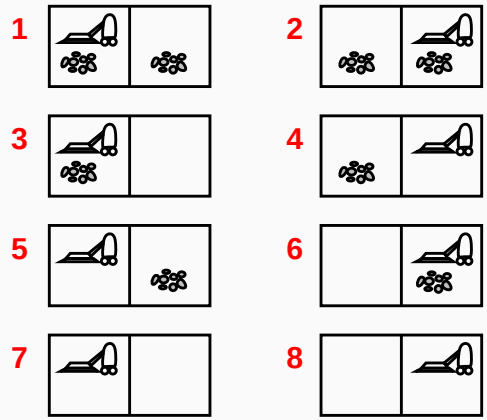
Single-state, start in #5. Solution?

$[Right, Suck]$

Conformant, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$

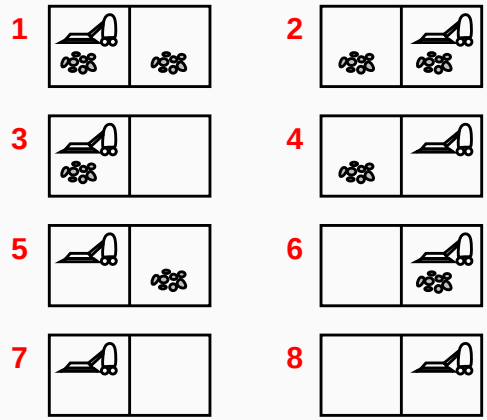
e.g., $Right$ goes to $\{2, 4, 6, 8\}$. Solution?

$[Right, Suck, Left, Suck]$



Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Vacuum world: different classes of problems



Single-state, start in #5. Solution?

$[Right, Suck]$

Conformant, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$

e.g., $Right$ goes to $\{2, 4, 6, 8\}$. Solution?

$[Right, Suck, Left, Suck]$

Conditional, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$

$Suck$ can dirty a clean carpet

Local sensing: dirt in current location. Solution?

Examples

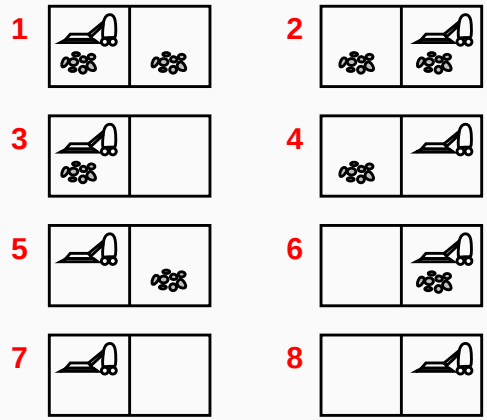
Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Vacuum world: different classes of problems



Single-state, start in #5. Solution?

$[Right, Suck]$

Conformant, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$

e.g., $Right$ goes to $\{2, 4, 6, 8\}$. Solution?

$[Right, Suck, Left, Suck]$

Conditional, start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$

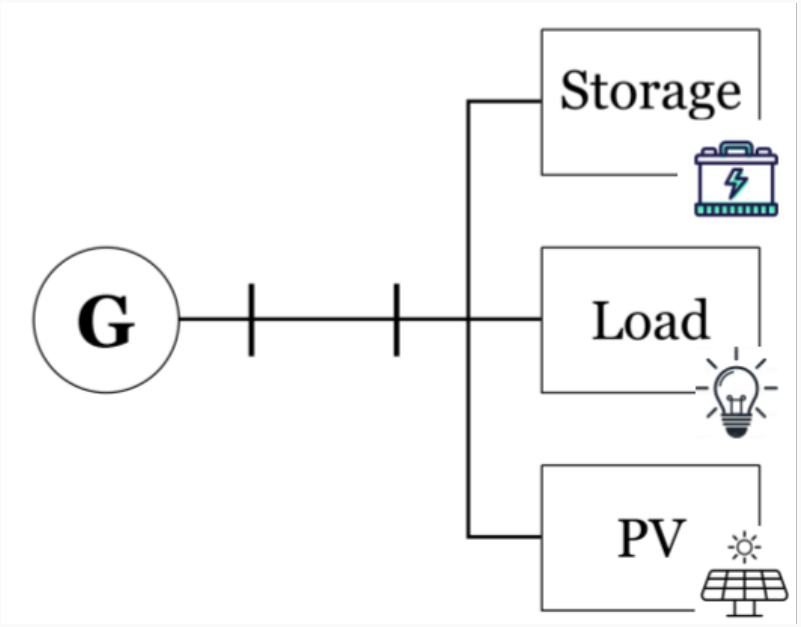
$Suck$ can dirty a clean carpet

Local sensing: dirt in current location. Solution?

$[Right, if\ dirt\ then\ Suck, Left, if\ dirt\ then\ Suck]$

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Storage Management



Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Stock Market Investment

Examples

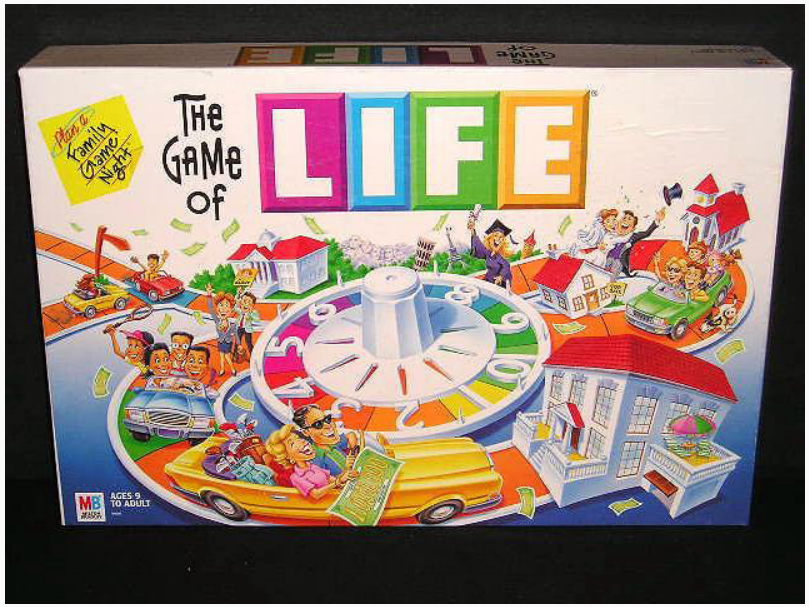
Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Life



Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Medical Procedure

Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Driving

Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Policies for Rational Agents

Policies for Rational Agents

Typically cannot be described explicitly for each state and require a compact representation.

An **optimal policy**, denoted π^* is *better* than or equal to all other policies.

Since we focus on rational agents, we typically have a clearly defined utility measure $\mathcal{U}$ which an optimal policy maximizes.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

From Models to Decision-Making

Given some assumptions about the model of the environment, the agent must compute a policy that maximizes utility.

The key questions become:

Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

From Models to Decision-Making

Given some assumptions about the model of the environment, the agent must compute a policy that maximizes **utility**.

The key questions become:

- What is the objective, and how to design a reward function that induces a utility-maximizing behavior?
- Do we know the dynamics?
- Can we acquire data from / simulate the environment?
- What can be observed?
- Is the state space small enough for exact computation?
- How much resources are available got computing a policy.
- ...

Different answers lead to different control algorithms.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Different Objectives

Reach a goal state.

- **Example**: Earn 500 NIS.

Stay in goal states indefinitely (infinite horizon).

- **Example:** Never allow a negative bank account balance.

Maximize the probability of reaching a goal state.

- **Example:** To be able to finance buying a house by 2035.

Eventually reach a goal state.

- **Example:** Winning the Nobel prize.

Collect the maximal *expected* rewards/minimal expected costs (infinite horizon).

- **Example:** Maximize your future income.

etc.

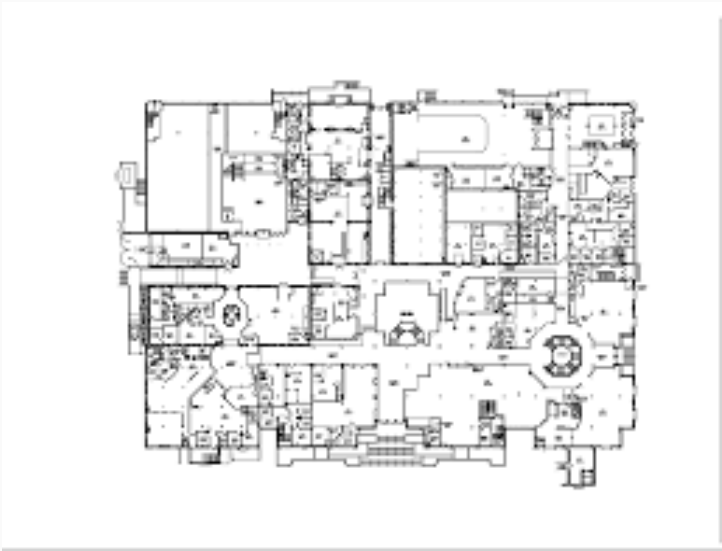
Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Reminder: Policy

- A **deterministic** policy is a mapping:
  - $\pi : \mathcal{S} \rightarrow \mathcal{A}$ from states to actions
  - $\pi : \mathcal{B} \rightarrow \mathcal{A}$ from beliefs to action
  - $\pi(s)/\ \pi(\beta)$ is the (single) action the agent will perform at state s or belief β.
- A **non-deterministic** policy is a mapping:
  - $\pi : \mathcal{S} \rightarrow \mathcal{A}^n$ from states to actions,
  - $\pi : \mathcal{B} \rightarrow \mathcal{A}^n$ from beliefs to actions
  - $\pi(s)/\ \pi(\beta)$ is the set of actions, one of which the agent will perform at state s or belief β.
- A **stochastic** policy is a mapping:
  - $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ from states to actions
  - $\pi : \mathcal{B} \times \mathcal{A} \rightarrow [0, 1]$ from beliefs to action
  - $\pi(s, a)/\ \pi(\beta, a)$ is the probability the agent will perform action a at state $s/$ belief β.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

How to come up with ’good’ policies?



How to come up with 'good' policies?

Given a model of the environment and an objective, how can an agent compute a good policy?

Common approaches include:

- **Search and planning** over states or beliefs
- **Dynamic programming** using Bellman equations
- **Approximate methods** for large state spaces
- **Learning-based methods** when the model is unknown or too complex

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Characterizing Decision-Making Algorithms

Formulating a Decision-Making Algorithm

We are going to explore various decision-making algorithms, that typically combine three ingredients:

- **Representation:** how states, actions, beliefs, rewards, or values are represented?
- **Evaluation:** how we estimate the future quality of states, actions, or policies?
- **Optimization:** how we improve decisions based on these estimates?

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Representation

How to model the decision problem?

- State representation:
  - explicit vs. factored (features, fluents)
  - belief states for partial observability
- Action representation:
  - discrete / continuous actions
  - structured actions (e.g., STRIPS operators)
- Model representation:
  - transition function $\mathcal{P}(s, a, s')$, reward $\mathcal{R}(s, a)$
  - explicit models vs. learned / implicit models
- Policy / value representation:
  - tabular vs. function approximation (e.g., neural networks)

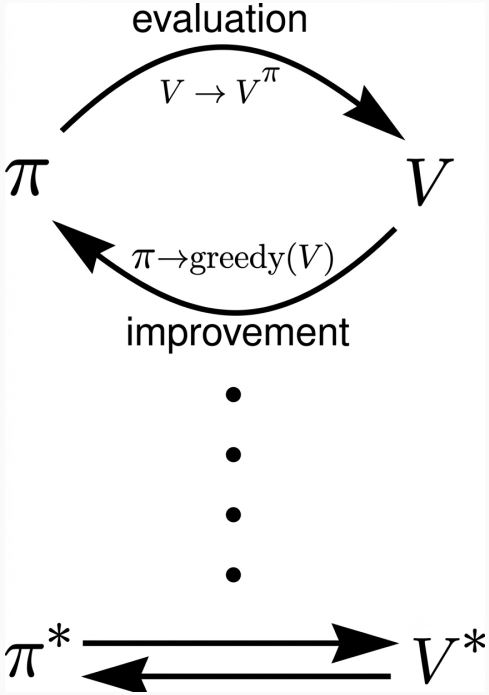
Abstraction: we often map a complex space to a simpler one.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Policy Evaluation and Policy Optimization

Prediction / Evaluation: evaluate the future given a policy - evaluate the quality of a given policy

Control/ Update / Improvement: optimize the policy.



Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Model Based vs. Model Free

A **model** typically refers to the transition function $\mathcal{P}$ and the reward function $\mathcal{R}$.

A **model free** approach only maintains a policy or value function, but no model.

A **model-based** approach maintains a policy or value function and a model.

What's the strength of each approach ?

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Value vs. Policy Based

Value Based

- No (explicit) policy is maintained
- Instead, a value function is maintained.

Policy Based

- A policy is explicitly maintained.

Combined approach (a.k.a Actor-Critic)

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

On Policy vs. Off Policy

We distinguish between the **behavior policy**, according to which an agent interacts with the environment and the **target policy**, the policy the agent learns that will optimize its utility.

On-policy methods evaluate and improve the policy that is used to make decisions.

Off-policy methods evaluate and improve a (target) policy different from the (behavior) policy that is used to generate the data.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Online vs. Offline

Online: iteratively collecting data (a.k.a. experiences) by interacting with the environment.

Offline: utilizing previously collected data, without additional online data collection.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Planning and Replanning

Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Planning: the task of coming up with a sequence of actions to achieve a goal.

- **Sensorless (conformant) planning:** constructing sequential plans to be executed without perception
- **Conditional (contingent) planning**: constructing a conditional plan with different branches for different contingencies that could happen.
- **Continuous planning:** a planner designed to persist over a lifetime.

Replanning: constructing a plan, but **monitoring its execution** and generating a new plan when necessary.

Solution Approaches

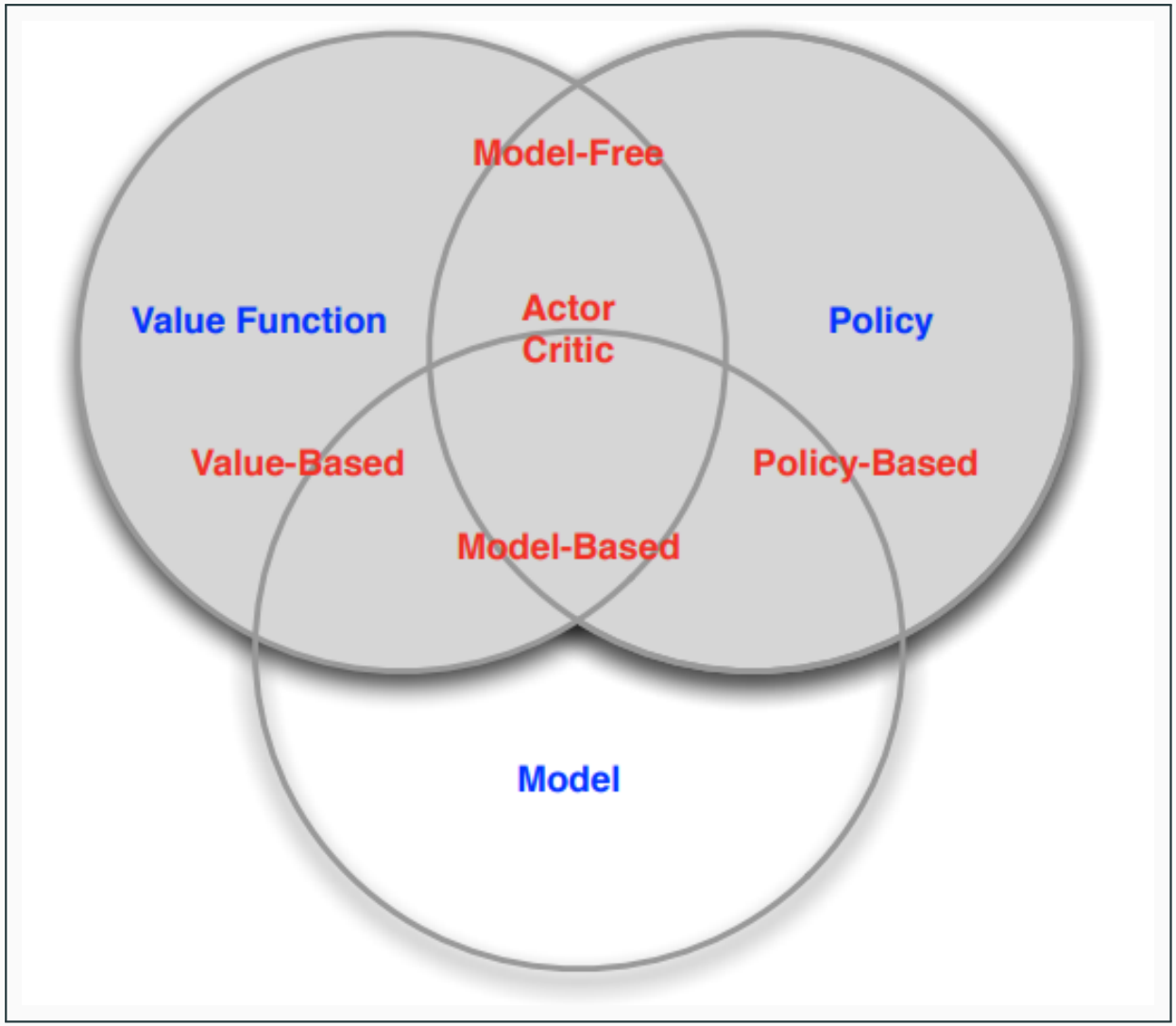


image by David Silver

Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Approaches to Decision-Making we Will Explore

Different assumptions lead to different algorithms.

We will explore:

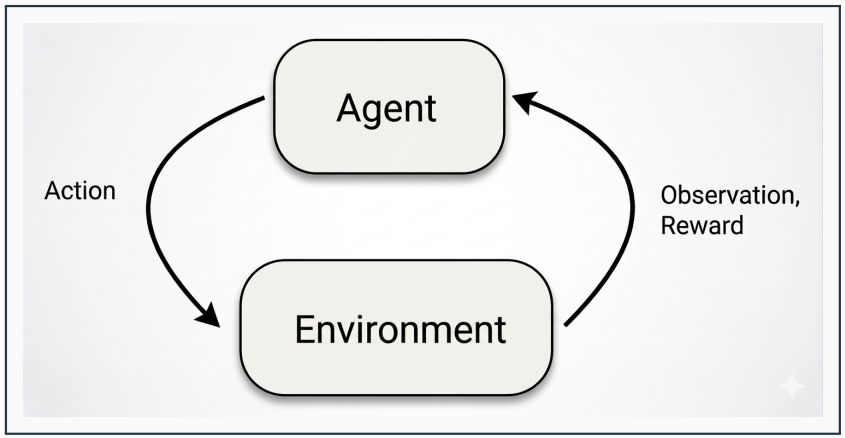
- **Supervised learning**
  - Imitation learning
- **Planning**
  - State space search and dynamic programming
  - Accounting for uncertainty and partial observability
- **Integrating planning and learning**
  - Monte-Carlo tree search
- **Reinforcement learning**
  - Value-based: TD learning
  - Policy-based: policy gradient
  - Combined approach: actor–critic
  - Model-based

Our objective is to learn how to choose the right approach.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Decision-Making Algorithms

Decision-Making Algorithms



Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms

Dimensions for evaluation

- **Completeness**: always find a solution if one exists?
- **Soundness***: if a solution is found, is it valid?
- **Time complexity**: number of nodes generated/expanded
- **Space complexity**: number of nodes stored in memory
- **Optimality**: does it always find a least-cost solution?
- **Anytime**: does solution quality improve with additional time or resources?

*In classical search/planning literature, soundness is often assumed implicitly because the transition model defines validity. It becomes more important when discussing approximate planners, learned planners, SAT/CSP solvers, theorem proving, or neural methods that may output invalid plans/actions.

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Anytime Algorithms

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Key properties

- **Interruptibility**: can be stopped at any time and still return a solution
- **Monotonic improvement**: best solution quality never degrades over time - additional computation or resources do not decrease the quality of the best solution found so far.
- **Performance profile**: quality as a function of resources

Why is this important?

- Enables graceful degradation when computation is limited
- Allows flexible allocation of resources based on urgency

Evaluating Algorithms: Examples

A* Search

- Completeness:
- Optimality:
- Time complexity:
- Space complexity:
- Anytime:

Monte Carlo Tree Search (MCTS)

- Completeness:
- Optimality:
- Time complexity:
- Space complexity:
- Anytime:

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Examples

A* Search

- **Completeness**: Yes (if branching factor is finite)
- **Optimality**: Yes (if heuristic is admissible)
- **Time complexity**: $\mathcal{O}(b^d)$
- **Space complexity**: $\mathcal{O}(b^d)$ (stores all frontier nodes)
- **Anytime**: No (returns solution only after full search)

Monte Carlo Tree Search (MCTS)

- **Completeness**:
- **Optimality**:
- **Time complexity**:
- **Space complexity**:
- **Anytime**:

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Examples

A* Search

- **Completeness**: Yes (if branching factor is finite)
- **Optimality**: Yes (if heuristic is admissible)
- **Time complexity**: $\mathcal{O}(b^d)$
- **Space complexity**: $\mathcal{O}(b^d)$ (stores all frontier nodes)
- **Anytime**: No (returns solution only after full search)

Monte Carlo Tree Search (MCTS)

- **Completeness**: No (finite time)
- **Optimality**: Asymptotically optimal (in the limit)
- **Time complexity**: $\mathcal{O}(N)$ simulations
- **Space complexity**: $\mathcal{O}(M)$ nodes stored
- **Anytime**: Yes (performance improves with more simulations)

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Modern Considerations

Traditional Approaches:

- Static, fully known environments
- Model is given (no data required)
- Uncertainty often ignored or simplified
- Offline computation

Focus on **worst-case guarantees**. Metrics: time, space, optimality

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Modern Considerations

Traditional Approaches:

- Static, fully known environments
- Model is given (no data required)
- Uncertainty often ignored or simplified
- Offline computation

Focus on **worst-case guarantees**. Metrics: time, space, optimality

Modern Approaches:

- Dynamic, uncertain environments
- Metrics: return, regret, risk, safety
- Data-driven (learning from interaction)
- Online, real-time decision-making

Focus on **empirical performance, robustness, and resource consumption**

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Modern Considerations

Solution quality: how good is the returned solution if optimality is not guaranteed?

Robustness: how sensitive is performance to noise, uncertainty, or modeling errors?

Scalability: how does performance change as the problem size grows?

Adaptivity: can the algorithm improve or adjust based on experience?

Generality: does the method apply broadly across domains or rely on strong assumptions?

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Modern Considerations

For evaluating **data-driven** methods, we consider:

Sample cost: how many resources (time, energy, money) are required to obtain a single sample?

Sample complexity: how many interactions are required to learn a good policy (to a desired level of performance)?

Regret: how much cumulative reward is lost compared to an optimal policy over time?

Convergence: does the algorithm converge, and at what rate?

Stability: how sensitive is learning to noise, randomness, or hyperparameters?

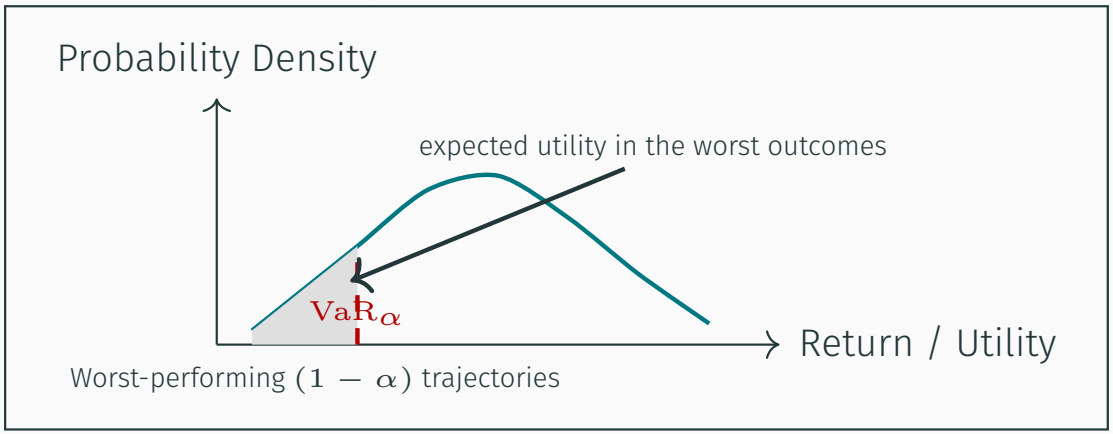
Exploration vs. exploitation: how effectively does the algorithm

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Modern Considerations

For evaluation under **uncertainty**:

- Performance must be evaluated **in expectation** or over multiple runs
- Metrics often include:
  - average return
  - variance / risk-sensitive measures
  - worst-case performance (robustness)



- Empirical evaluation becomes essential:
  - simulations
  - benchmarks
  - ablations and sensitivity analysis

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Additional requirements in modern settings

For evaluation under risk and hard constraints, we need to consider:

Safety: does the algorithm avoid unsafe or catastrophic outcomes during execution or learning?

Risk-awareness: does it account for variability, rare events, or worst-case performance (not just expectation)?

Uncertainty-awareness: does it explicitly reason about partial observability and model uncertainty?

Data efficiency: how effectively does it learn from limited data or interactions?

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

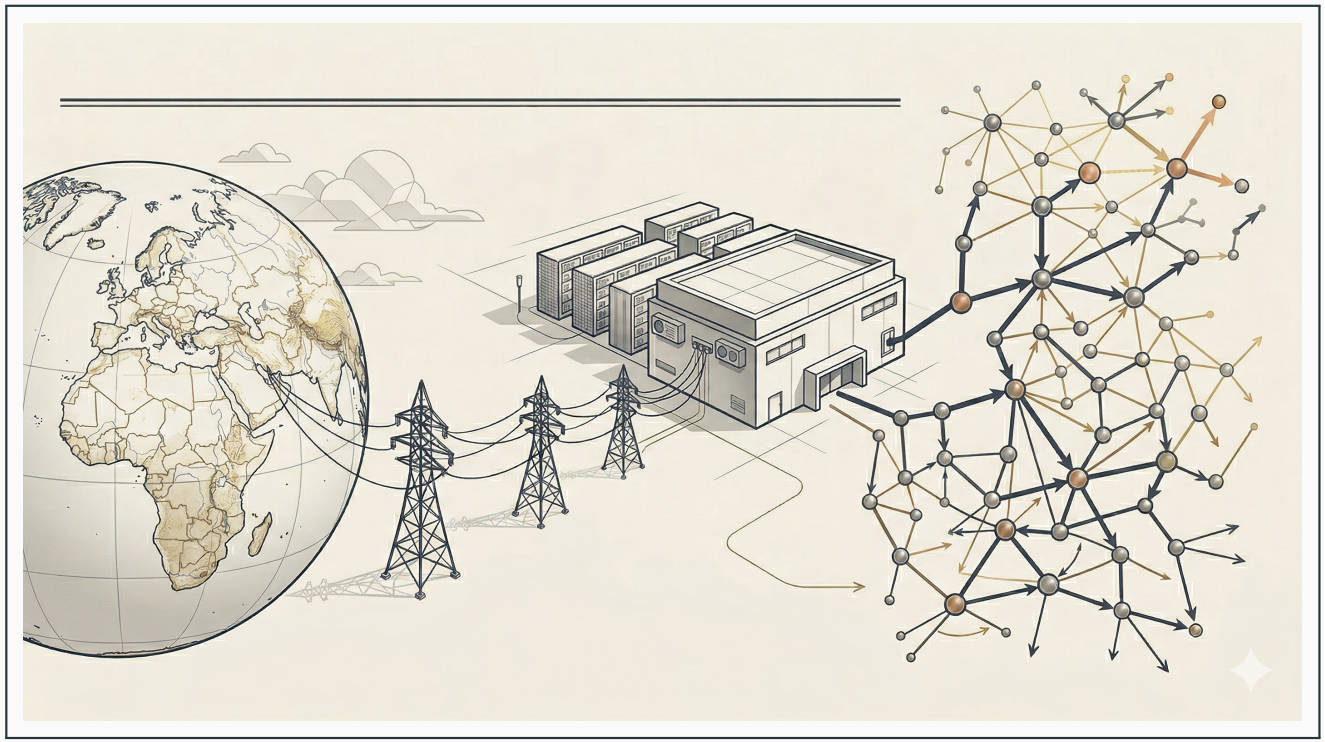
Additional requirements in modern settings

To support real-world settings, we need to consider

- **Interpretability**: can the decisions be explained or understood by humans?
- **Coordination**: can multiple agents effectively collaborate?
- **Communication**: how much information exchange is required between agents?
- **Robustness to others**: does performance degrade gracefully when other agents behave unpredictably?
- **Sim-to-real transfer**: does performance carry over from simulation to real-world deployment?

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Modern Considerations



Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Evaluating Algorithms: Modern Considerations

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Modern algorithms must address **resource scarcity** and **resource volatility**, such as fluctuating renewable energy inputs or dynamic load conditions.

Under such constraints, we must consider **energy efficiency**: the total energy consumed during computation, including processing, memory access, and system-level overhead (e.g., cooling and communication).

Energy-aware algorithms are designed to:

- minimize power consumption
- adapt to changing resource availability
- trade off performance with resource usage

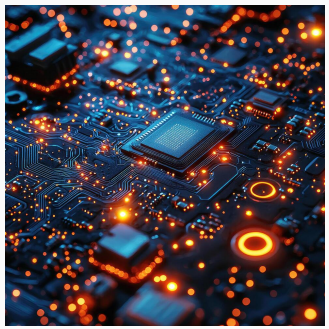
Evaluating Algorithms: Modern Considerations

Resource-aware and adaptive decision-making algorithms explicitly reason about trade-offs between performance, uncertainty, and resource consumption.

This paradigm is often studied under the umbrella of **Green AI**.

Energy-based metrics include:

- Joules per operation
- Total energy (kWh)
- Carbon footprint (CO_2 emissions)



Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Energy-Aware Algorithm Design: Examples

Two operations may share the same asymptotic complexity, but have vastly different energy and carbon footprints.

Example: integer addition vs. tensor inference

Integer addition:

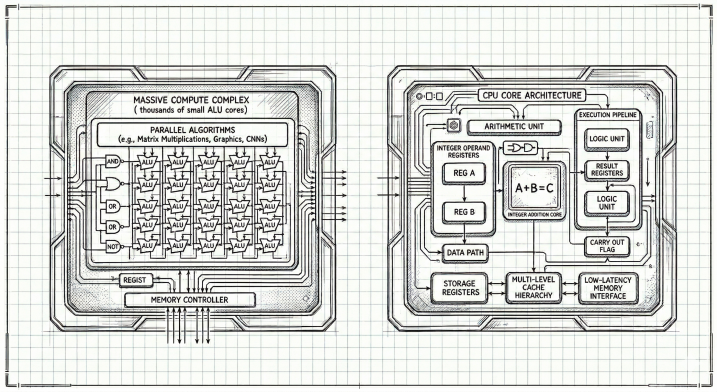
- CPU operation
- Executed within existing CPU cycles
- Low incremental energy
- Minimal thermal impact

Tensor inference:

- GPU/TPU operation
- Requires data transfer and hardware activation
- High peak power usage
- May trigger cooling systems

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Energy-Aware Algorithm Design: Examples



Energy depends on various factors such as:

- data movement
- hardware utilization
- cooling overhead (e.g., PUE in data centers)

Implication: An $O(n)$ CPU-based solution may be **more energy-efficient** than an $O(1)$ accelerator call

Examples

Policies for Rational Agents

Characterizing Decision-Making Algorithms

Evaluating Decision-Making Algorithms

Use Cases

Evaluating Algorithms: Conclusion

There is typically no universally best algorithm.

Trade-offs arise between:

- time vs. space
- optimality vs. efficiency
- generality vs. specialization
- energy efficiency vs. speed
- etc.

The choice of algorithm depends on:

- problem structure (e.g., branching factor, depth)
- available computational resources
- requirements on solution quality and guarantees

Classical metrics focus on computational properties. Modern systems must also satisfy operational and interaction

Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Evaluating Algorithms: Conclusion

Consider algorithms you have encountered (e.g., BFS, DFS, A*, Dijkstra, gradient Decent, RL methods).

For each algorithm, ask:

- Is it **complete**? Is it **optimal**?
- What are its **time and space requirements**?
- Does it exhibit **anytime behavior**?
- How does it use **resources** (computation, memory, data)?
- Can it handle **uncertainty** or only deterministic settings?

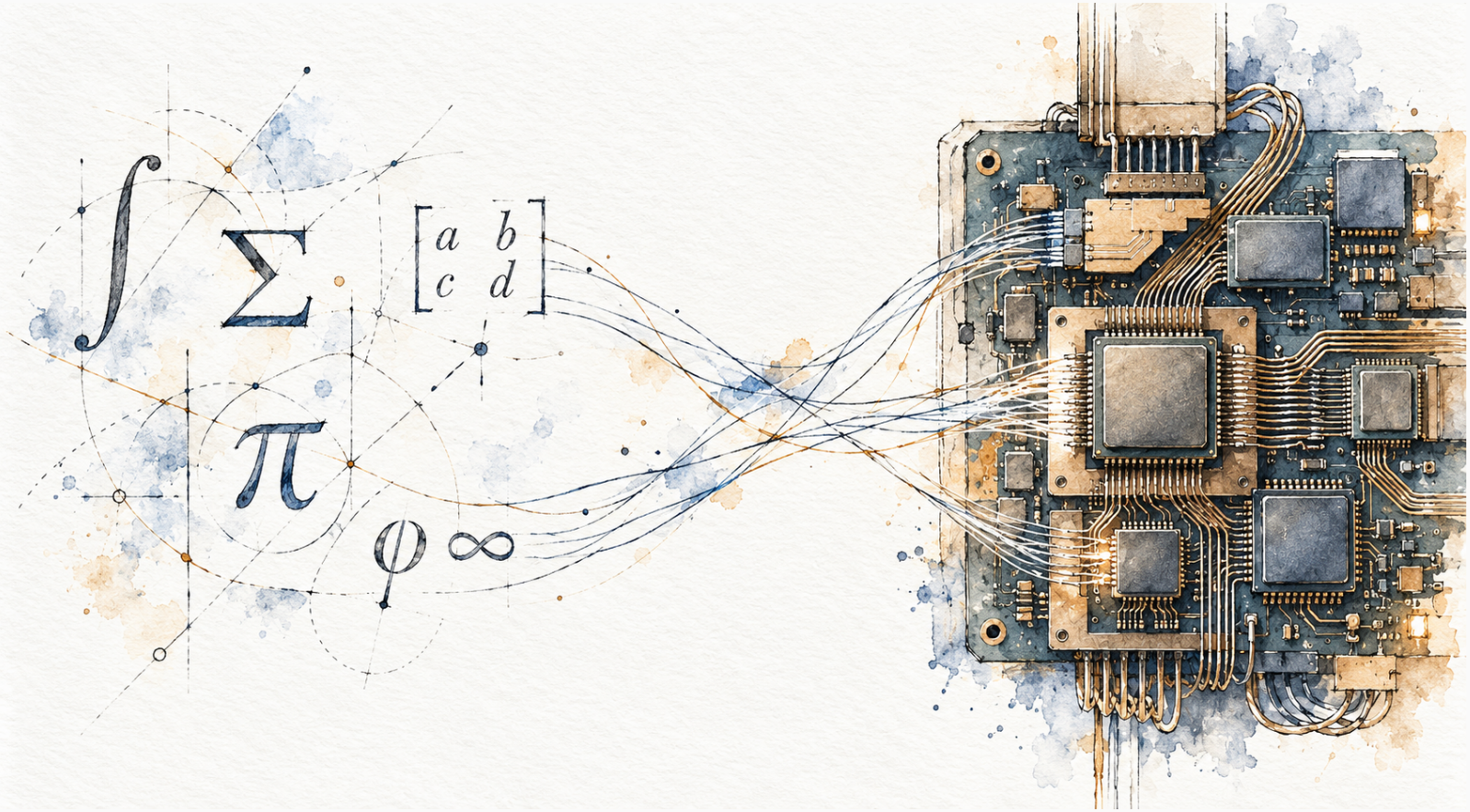
Which algorithm would you choose under:

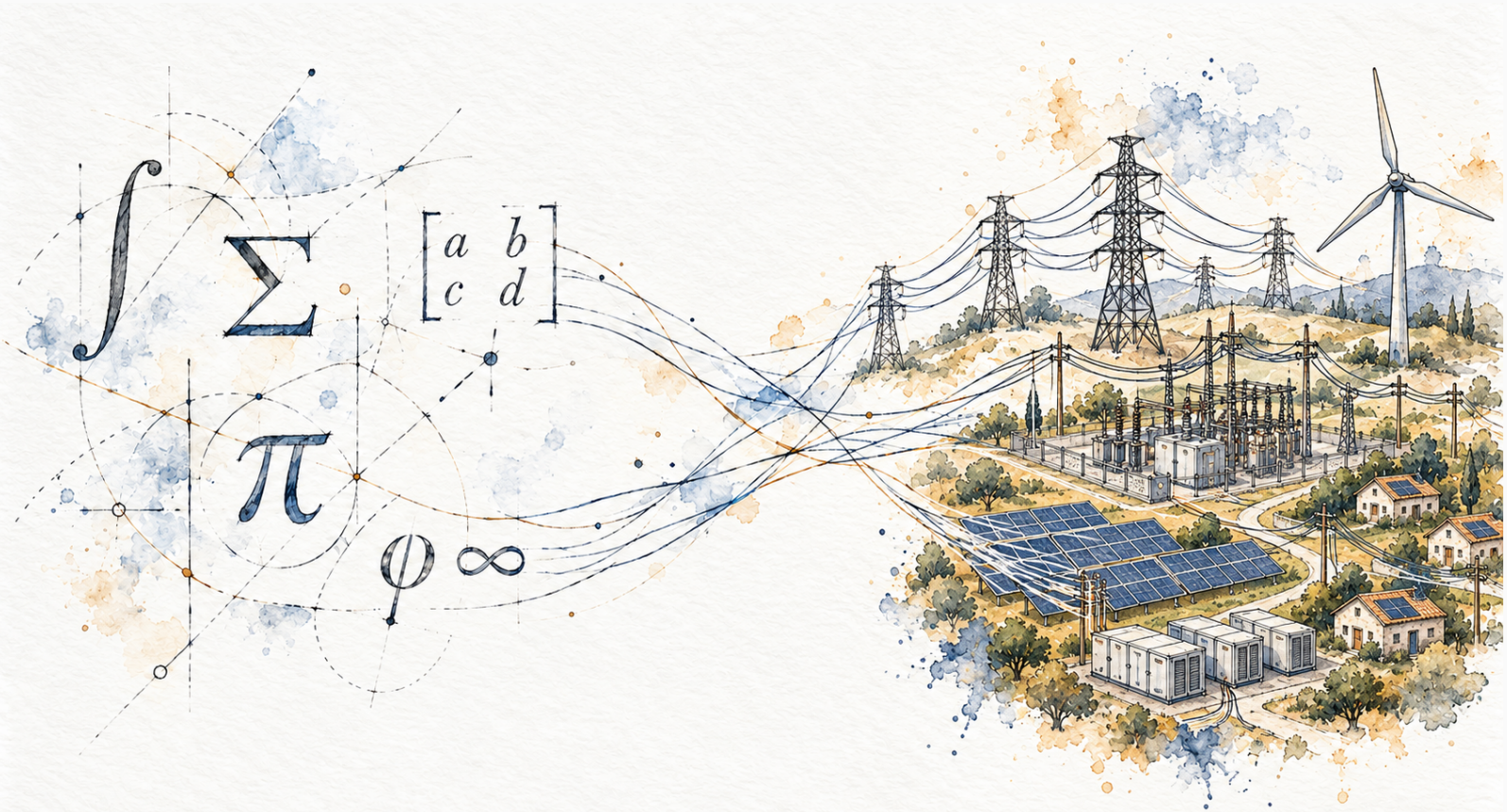
- strict time constraints?
- limited memory?
- uncertain or dynamic environments?

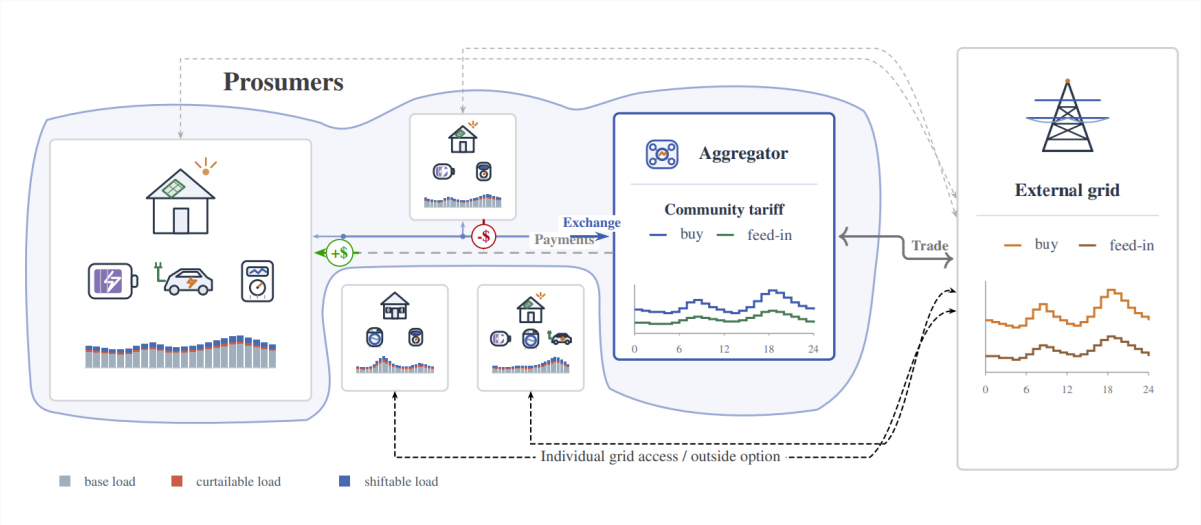
Goal: Develop intuition for selecting the right algorithm for the right problem.

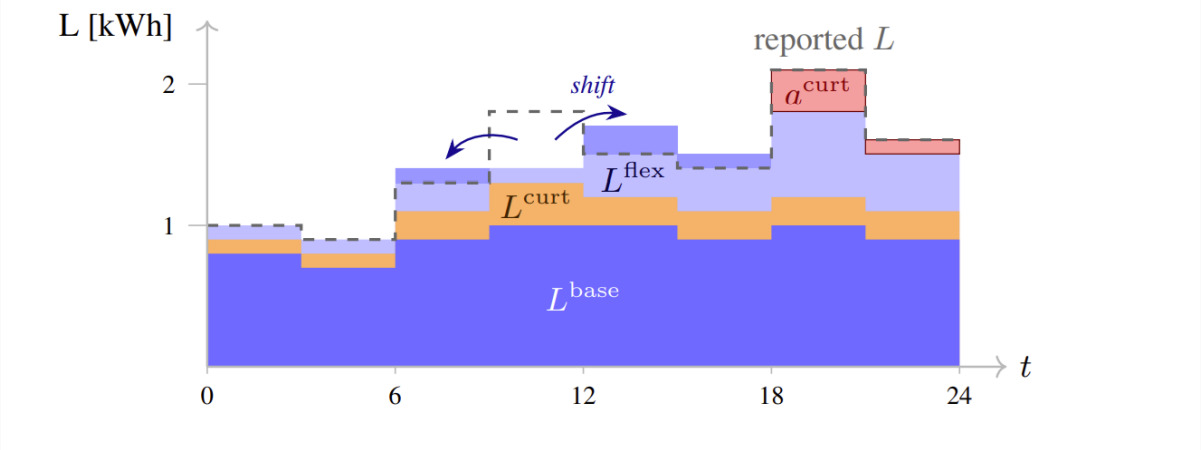
Examples
Policies for Rational Agents
Characterizing Decision-Making Algorithms
Evaluating Decision-Making Algorithms
Use Cases

Use Cases






Prosumers
Aggregator
Community tariff
buy
feed-in
Exchange
Payments
Trade
External grid
buy
feed-in
Individual grid access / outside option
base load
curtailable load
shiftable load


L [kWh]
reported L
shift
a^{curt}
L^{flex}
L^{curt}
L^{base}
t
0
6
12
18
24
1
2